1. WAP in C to create a BST that includes the following functions:
a) insert_node
b) delete_node
c) find_min
d) find_max
e) InOrder, PreOrder, PostOrder
Using the main function, show the working of all these functions.

Source Code:

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left_child;
    struct node *right_child;
};

struct node *create_node(int value){
    struct node *new;
    new = (struct node *) malloc(sizeof(struct node));
    new->data = value;
    new->left_child = NULL;
    new->right_child = NULL;
    return new;
}

struct node* insert_node(struct node *root, int value){
    if (root == NULL){
        return create_node(value);
    }
    if (value < root->data){
        root->left_child = insert_node(root->left_child, value);
    } else if (value > root->data){
        root->right_child = insert_node(root->right_child, value);
    }
    return root;
}

struct node* find_min(struct node *root){
    if (root == NULL){
        return NULL;
    }
    while (root->left_child != NULL){
        root = root->left_child;
    }
    return root;
}

struct node* find_max(struct node *root){
    if (root == NULL){
        return NULL;
```

```
    }
    while (root->right_child != NULL){
        root = root->right_child;
    }
    return root;
}

struct node* delete_node(struct node *root, int value){
    if (root == NULL){
        printf("Node not found\n");
        return NULL;
    }
    if (value < root->data){
        root->left_child = delete_node(root->left_child, value);
    } else if (value > root->data){
        root->right_child = delete_node(root->right_child, value);

    } else {
        if (root->left_child == NULL && root->right_child == NULL){
            free(root);
            return NULL;
        }
        else if (root->left_child == NULL){
            struct node *temp = root->right_child;
            free(root);
            return temp;
        } else if (root->right_child == NULL){
            struct node *temp = root->left_child;
            free(root);
            return temp;
        }
        else {
            struct node *temp = find_min(root->right_child);
            root->data = temp->data;
            root->right_child = delete_node(root->right_child, temp->data);
        }
    }
    return root;
}

void InOrder(struct node *ptr){
    if(ptr == NULL){
        return;
    }
    InOrder(ptr->left_child);
    printf("%d ", ptr->data);
    InOrder(ptr->right_child);
}

void PreOrder(struct node *ptr){
    if(ptr == NULL){
        return;
    }
    printf("%d ", ptr->data);
```

```
    PreOrder(ptr->left_child);
    PreOrder(ptr->right_child);
}

void PostOrder(struct node *ptr){
    if(ptr == NULL){
        return;
    }
    PostOrder(ptr->left_child);
    PostOrder(ptr->right_child);
    printf("%d ", ptr->data);
}

int main(){
    struct node *root = NULL;

    root = insert_node(root, 20);
    root = insert_node(root, 10);
    root = insert_node(root, 30);
    root = insert_node(root, 5);
    root = insert_node(root, 15);
    root = insert_node(root, 25);
    root = insert_node(root, 35);

    root = delete_node(root, 20);
    root = delete_node(root, 10);

    printf("Minimum Element: %d\n", find_min(root)->data);
    printf("Maximum Element: %d\n", find_max(root)->data);

    printf("InOrder Traversal: ");
    InOrder(root);
    printf("\n");

    printf("PreOrder Traversal: ");
    PreOrder(root);
    printf("\n");

    printf("PostOrder Traversal: ");
    PostOrder(root);
    printf("\n");
    return 0;
}
```

Output:

```
PS D:\DS\LAB8> gcc p1.c -o p1.exe
PS D:\DS\LAB8> .\p1.exe
Minimum Element: 5
Maximum Element: 35
InOrder Traversal: 5 15 25 30 35
PreOrder Traversal: 25 15 5 30 35
PostOrder Traversal: 5 15 35 30 25
PS D:\DS\LAB8>
```